1. 今日の位置づけ（第3回との分担）

第3回では DAO で **SELECT（参照）** を `PreparedStatement` により安全に扱いました。

第4回では同じく DAO に閉じて **INSERT / UPDATE / DELETE（更新系）** を実行し、複数の更新を **1つのトランザクション**にまとめる方法を扱います。

題材は引き続き **lesson** DB の **emps**（および既存の **depts**）。Java サンプルは **Unit03_DAO/src** の `DaoBase` / `EmpDto` / `EmpDao` をそのまま使います。トランザクションの実行例は **Unit04_DAO_Update/src/TransactionSample.java** を参照してください。

2. 更新系の型（executeUpdate と更新件数）

- 参照は `executeQuery()` → `ResultSet`。
- 更新系は `executeUpdate()` → **int（影響した行数）**。
- `EmpDao#insert` / `updateSalaryAndDept` / `delete` はいずれも `PreparedStatement` で `?` に値をバインドする（第3回と同じ原則）。

3. トランザクションとは（なぜまとめるか）

複数の SQL を順に実行するとき、途中でエラーが出ると **前半だけ反映された中途半端なデータ**が残ることがあります。

トランザクションでは、成功時にまとめて `commit`（確定）、失敗時に `rollback`（取り消し）し、**全部成功か、全部なかったこと**のどちらかにそろえます。

JDBC のお約束（この講義で使う型）

- **同じ `Connection`** に対して複数の `PreparedStatement` を実行する。
- 開始前に `setAutoCommit(false)` とする（自動コミットを止める）。
- `try { ... con.commit(); } catch (SQLException e) { con.rollback(); throw e; } finally { con.setAutoCommit(true); }`
- `EmpDao` の各メソッドは内部で `getConnection()` しているため **トランザクションに使えない**。発展課題では **1本の `Connection` を取り、SQL を直接並べる**か、`Connection` を引数に取る DAO メソッドを自分で追加する。

4. 穴埋め〜トランザクションの骨格

下線部（____）を埋めてから、解答を開いてください。

```
try (Connection con = getConnection()) {
        con.setAutoCommit( (1) );
        try {
                // 複数の PreparedStatement 実行
                con.(2)();
        } catch (SQLException e) {
                con.(3)();
                // 教材ではまず原因が見えるようにする（本来はログ設計・通知・リカバリー
                e.printStackTrace();
        } finally {
                con.setAutoCommit( (4) );
        }
}
```

(1)〜(4) に入る語を答えよ（Java のリテラルまたはメソッド名）。

▼ 解答

```
(1) false
(2) commit
(3) rollback
(4) true
```

4-2. 理解チェック（少しだけ難しめ）

問題　次の問いに答えよ（口頭でOK）。

- **(1)** なぜ `finally` で `setAutoCommit(true)` に戻す必要がある？（1行）
- **(2)** `commit` は「いつ」呼ぶ？ `rollback` は「いつ」呼ぶ？（それぞれ1行）
- **(3)** なぜ「複数の更新をまとめる」には **同じ Connection** が必要？（1行）
- **(4)** 次のうち、更新系の実行メソッドとして正しいのはどれ？\n+
  \n+
  - **A** executeQuery
    \n+
  - **B** executeUpdate
    \n+
  - **C** next
    \n+

\n+

▼ **ヒント**

(1) は「次の処理まで自動コミットがOFFのままだとどうなる？」を想像。 (3) は「トランザクションの単位は何か？」を思い出す。

▼ **解答例**

- **(1)** 例：自動コミットがOFFのままだと、次に実行する更新が意図せず同じトランザクションに入ったり、commitし忘れで反映されないなど事故が起きるため。
- **(2)** commit：全部の更新が成功したとき。 rollback：途中で失敗（SQLExceptionなど）したとき。
- **(3)** トランザクションは「同じConnectionで行った操作」をまとめる仕組みで、Connectionが違うと別トランザクションになるから。
- **(4)** B

5. EmpDao の更新系（ドラッグでコピー）

▼ insert / updateSalaryAndDept / delete（概要）

全文は **Unit03_DAO/src/EmpDao.java** にあります。主キー `emp_id` は既存データと重複しない値（演習では 11 以降など）を使うこと。

マウスで全選択して貼り付け欄へドラッグしてください：

```
// insert
String sql = "INSERT INTO emps (emp_id, fname, lname, salary, dept_no) VALUES (?,
?, ?, ?, ?)";
// update（給与と部署）
sql = "UPDATE emps SET salary = ?, dept_no = ? WHERE emp_id = ?";
// delete
sql = "DELETE FROM emps WHERE emp_id = ?";
```

6. 例外処理と rollback

第3回前の例外教材（`Unit90_Exception`）で try / catch の流れを押さえたうえで、`catch` 内で `rollback` を呼ぶと「失敗時に取り消す」がコード上も追いやすくなります。